

demo_report

December 3, 2025

1 LLM Cost Decomposition Analysis

Comprehensive analysis of multi-stage pipelines, agentic workflows, A/B testing, and security document analysis.

1.1 Analyses:

1. **Pipeline Cost Comparison** - Compare costs across pipeline types
2. **Stage-Level Attribution** - Where does the money go?
3. **Model Comparison** - Flash vs Pro economics
4. **Agentic Patterns** - ReAct iterations, multi-turn growth
5. **Streaming Metrics** - TTFT and throughput analysis
6. **A/B Testing** - Prompt variant performance
7. **Document Analysis** - Vulnerability detection accuracy
8. **Cost-Quality Tradeoffs** - Value optimization

```
[1]: import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from scipy import stats
import json

import sys
sys.path.insert(0, '..')

# Database queries
from src.db import (
    get_runs,
    get_stages,
    get_quality_scores,
    get_pipeline_summary,
    get_stage_summary,
    get_cost_by_stage_type,
    get_cost_by_model,
    get_iteration_analysis,
    get_context_growth_analysis,
```

```

    get_streaming_analysis,
    # A/B test queries
    get_ab_test_summary,
    get_ab_test_quality,
    get_ab_test_cost_quality_ratio,
    get_ab_tests,
)

# Vulnerability ground truth
from src.evaluation import (
    get_vulnerabilities,
    get_all_document_ids,
    count_by_severity,
    calculate_detection_score,
)

from src.utils import format_cost

# Plotly theme
import plotly.io as pio
pio.templates.default = 'plotly_white'

```

1.2 1. Load Data

```

[2]: runs_df = get_runs(success_only=True)
     stages_df = get_stages()
     quality_df = get_quality_scores()

     print(f" Dataset Overview")
     print(f"{' '*40}")
     print(f"Total pipeline runs: {len(runs_df):,}")
     print(f"Total stage executions: {len(stages_df):,}")
     print(f"Quality evaluations: {len(quality_df):,}")

     if len(runs_df) > 0:
         print(f"\nPipeline types: {runs_df['pipeline_type'].unique().tolist()}")
         print(f"Workflows: {runs_df['workflow'].unique().tolist()}")
         print(f"Models: {runs_df['model'].unique().tolist()}")
         print(f"\nTotal cost: {format_cost(runs_df['total_cost'].sum())}")

```

```

Dataset Overview
=====
Total pipeline runs: 736
Total stage executions: 1,753
Quality evaluations: 735

```

```

Pipeline types: ['linear', 'react', 'multiturn', 'self_correcting', 'ab_test']
Workflows: ['verbosity', 'context', 'react', 'multiturn', 'self_correcting',

```

```
'document', 'ab_test']
Models: ['gemini-2.5-flash', 'gemini-2.5-pro']
```

Total cost: \$8.07

```
[3]: # Pipeline summary
pipeline_summary = get_pipeline_summary()
if not pipeline_summary.empty:
    print("Pipeline Summary:")
    display(pipeline_summary)
```

Pipeline Summary:

	pipeline	pipeline_type	model	runs	avg_cost	\
0	ab_generation	ab_test	gemini-2.5-flash	80	0.000909	
1	ab_generation	ab_test	gemini-2.5-pro	56	0.006824	
2	context_long	linear	gemini-2.5-flash	20	0.002015	
3	context_long	linear	gemini-2.5-pro	20	0.014181	
4	context_short	linear	gemini-2.5-flash	20	0.000150	
5	context_short	linear	gemini-2.5-pro	20	0.001126	
6	doc_analysis_hybrid	linear	gemini-2.5-flash	20	0.017726	
7	doc_analysis_hybrid	linear	gemini-2.5-pro	20	0.017599	
8	doc_analysis_iterative	linear	gemini-2.5-flash	20	0.014313	
9	doc_analysis_iterative	linear	gemini-2.5-pro	20	0.029987	
10	doc_analysis_simple	linear	gemini-2.5-flash	20	0.002213	
11	doc_analysis_simple	linear	gemini-2.5-pro	20	0.014771	
12	doc_analysis_thorough	linear	gemini-2.5-flash	20	0.016729	
13	doc_analysis_thorough	linear	gemini-2.5-pro	20	0.046724	
14	hybrid_cot	linear	gemini-2.5-flash	20	0.011129	
15	hybrid_cot	linear	gemini-2.5-pro	20	0.011335	
16	multiturn_3	multiturn	gemini-2.5-flash	20	0.004425	
17	multiturn_3	multiturn	gemini-2.5-pro	20	0.034753	
18	multiturn_5	multiturn	gemini-2.5-flash	20	0.007029	
19	multiturn_5	multiturn	gemini-2.5-pro	20	0.065920	
20	react_hybrid	react	gemini-2.5-flash	20	0.011356	
21	react_hybrid	react	gemini-2.5-pro	20	0.010286	
22	react_research	react	gemini-2.5-flash	20	0.000467	
23	react_research	react	gemini-2.5-pro	20	0.000491	
24	self_correcting	self_correcting	gemini-2.5-flash	20	0.001791	
25	self_correcting	self_correcting	gemini-2.5-pro	20	0.001602	
26	self_correcting_hybrid	self_correcting	gemini-2.5-flash	20	0.006068	
27	self_correcting_hybrid	self_correcting	gemini-2.5-pro	20	0.004661	
28	verbosity_concise	linear	gemini-2.5-flash	20	0.000035	
29	verbosity_concise	linear	gemini-2.5-pro	20	0.000290	
30	verbosity_cot	linear	gemini-2.5-flash	20	0.003580	
31	verbosity_cot	linear	gemini-2.5-pro	20	0.028255	

	avg_latency_ms	avg_input_tokens	avg_output_tokens	avg_iterations
0	17648.450000	23.300000	1508.512500	1.00

1	30958.446429	23.857143	1358.928571	1.00
2	41828.250000	2707.400000	2682.150000	1.00
3	70124.450000	2471.900000	2218.250000	1.00
4	9341.500000	261.250000	184.400000	1.00
5	21119.350000	252.450000	162.150000	1.00
6	95427.500000	3826.500000	8267.050000	1.00
7	95151.250000	3931.050000	8134.850000	1.00
8	97717.700000	4324.650000	6938.200000	1.00
9	123374.750000	3506.400000	5120.850000	1.00
10	37953.000000	2080.450000	3168.550000	1.00
11	63401.700000	1834.600000	2495.600000	1.00
12	145486.800000	6073.000000	11455.650000	1.00
13	176434.800000	5457.000000	7980.400000	1.00
14	79325.600000	3297.150000	5532.350000	1.00
15	80295.900000	3409.350000	5527.000000	1.00
16	55998.150000	9657.350000	4960.300000	1.00
17	102764.100000	9315.650000	4621.750000	1.00
18	68674.500000	24520.400000	5584.900000	1.00
19	145811.500000	27733.500000	6250.600000	1.00
20	50993.650000	2672.750000	2096.500000	2.05
21	46863.550000	2038.600000	2030.350000	1.80
22	8200.800000	74.000000	760.150000	1.00
23	7612.450000	74.000000	799.000000	1.00
24	35027.450000	2256.300000	2420.400000	1.15
25	30418.300000	1963.350000	2178.650000	1.10
26	45658.300000	2300.050000	2328.950000	1.15
27	38420.800000	1839.450000	1839.450000	1.15
28	3340.600000	20.800000	53.450000	1.00
29	13838.700000	20.800000	52.700000	1.00
30	60511.050000	3132.700000	5183.050000	1.00
31	111288.350000	3069.000000	4883.650000	1.00

1.3 2. Pipeline Cost Comparison

```
[4]: if not runs_df.empty:
    # Cost by pipeline and model
    cost_comparison = runs_df.groupby(['pipeline', 'model']).agg({
        'total_cost': ['mean', 'std', 'count'],
        'total_latency_ms': 'mean'
    }).round(6)
    cost_comparison.columns = ['avg_cost', 'std_cost', 'runs', 'avg_latency_ms']
    cost_comparison = cost_comparison.reset_index()

    fig = px.bar(
        cost_comparison,
        x='pipeline',
        y='avg_cost',
```

```

        color='model',
        barmode='group',
        error_y='std_cost',
        title='Average Cost by Pipeline and Model',
        labels={'avg_cost': 'Average Cost ($)', 'pipeline': 'Pipeline'},
        color_discrete_map={'gemini-2.5-flash': '#4ecdc4', 'gemini-2.5-pro': '#ff6b6b'}
    )
    fig.update_layout(xaxis_tickangle=-45)
    fig.show()

```

```

[5]: if not runs_df.empty:
    # Cost distribution boxplot
    fig = px.box(
        runs_df,
        x='pipeline',
        y='total_cost',
        color='model',
        title='Cost Distribution by Pipeline',
        labels={'total_cost': 'Total Cost ($)', 'pipeline': 'Pipeline'}
    )
    fig.update_layout(xaxis_tickangle=-45)
    fig.show()

```

1.4 3. Stage-Level Cost Attribution

```

[6]: # Cost by stage type
stage_costs = get_cost_by_stage_type()
if not stage_costs.empty:
    fig = px.sunburst(
        stage_costs,
        path=['model', 'stage_type'],
        values='total_cost',
        title='Cost Distribution by Stage Type and Model'
    )
    fig.show()

    display(stage_costs)

```

	stage_type	model	total_cost	avg_cost	count
0	conversation	gemini-2.5-pro	2.013461	0.012584	160
1	generation	gemini-2.5-pro	1.654240	0.008440	196
2	critique	gemini-2.5-pro	0.901604	0.009016	100
3	refinement	gemini-2.5-pro	0.837070	0.013951	60
4	evaluation	gemini-2.5-pro	0.478956	0.007983	60
5	extraction	gemini-2.5-pro	0.445604	0.005570	80
6	thinking	gemini-2.5-pro	0.419532	0.005448	77
7	refinement	gemini-2.5-flash	0.354530	0.002348	151

8	generation	gemini-2.5-flash	0.304209	0.001014	300
9	conversation	gemini-2.5-flash	0.229074	0.001432	160
10	validation	gemini-2.5-pro	0.172903	0.003759	46
11	extraction	gemini-2.5-flash	0.082215	0.000685	120
12	summarization	gemini-2.5-pro	0.080184	0.002005	40
13	validation	gemini-2.5-flash	0.024699	0.000549	45
14	thinking	gemini-2.5-flash	0.019153	0.000479	40
15	critique	gemini-2.5-flash	0.017386	0.000869	20
16	evaluation	gemini-2.5-flash	0.015054	0.000753	20
17	action	gemini-2.5-flash	0.013311	0.000350	38
18	summarization	gemini-2.5-flash	0.011790	0.000295	40

```
[7]: if not stages_df.empty:
      # Stage type breakdown
      stage_summary = stages_df.groupby('stage_type').agg({
          'cost': ['sum', 'mean', 'count'],
          'input_tokens': 'mean',
          'output_tokens': 'mean'
      }).round(4)
      stage_summary.columns = ['total_cost', 'avg_cost', 'count', 'avg_input_tokens', 'avg_output_tokens']
      stage_summary = stage_summary.sort_values('total_cost', ascending=False)
      display(stage_summary)
```

stage_type	total_cost	avg_cost	count	avg_input_tokens \
conversation	2.2425	0.0070	320	4451.6812
generation	1.9584	0.0039	496	390.8145
refinement	1.1916	0.0056	211	1599.7630
critique	0.9190	0.0077	120	1997.2417
extraction	0.5278	0.0026	200	479.8000
evaluation	0.4940	0.0062	80	1416.3500
thinking	0.4387	0.0037	117	623.1880
validation	0.1976	0.0022	91	1556.8132
summarization	0.0920	0.0011	80	523.8875
action	0.0133	0.0004	38	638.7895

stage_type	avg_output_tokens
conversation	1338.5969
generation	1591.5302
refinement	3193.8957
critique	1244.8083
extraction	1010.7950
evaluation	1156.9250
thinking	834.1966
validation	443.1648
summarization	315.0875

action

424.1842

1.5 4. Model Economics: Flash vs Pro

```
[8]: model_costs = get_cost_by_model()
if not model_costs.empty:
    display(model_costs)

    # Cost per 1K tokens comparison
    model_costs['cost_per_1k_input'] = model_costs['total_cost'] / \
    ↪(model_costs['total_input_tokens'] / 1000)
    model_costs['cost_per_1k_output'] = model_costs['total_cost'] / \
    ↪(model_costs['total_output_tokens'] / 1000)

    fig = px.bar(
        model_costs,
        x='model',
        y=['total_cost'],
        title='Total Cost by Model',
        labels={'value': 'Cost ($)', 'model': 'Model'}
    )
    fig.show()
```

	model	runs	total_cost	avg_cost	total_input_tokens \
0	gemini-2.5-flash	380	2.053215	0.005403	1345959
1	gemini-2.5-pro	356	6.021760	0.016915	1339678

	total_output_tokens
0	1353002
1	1162005

1.6 5. Agentic Workflow Analysis

```
[9]: # ReAct iteration analysis
iteration_data = get_iteration_analysis()
if not iteration_data.empty:
    print("ReAct & Self-Correcting Iteration Analysis:")
    display(iteration_data)

    # Termination reason distribution
    fig = px.pie(
        iteration_data,
        values='runs',
        names='termination_reason',
        title='Termination Reasons Distribution'
    )
    fig.show()
```

ReAct & Self-Correcting Iteration Analysis:

	pipeline	model	runs	avg_iterations	\
0	react_hybrid	gemini-2.5-flash	19	1.894737	
1	react_hybrid	gemini-2.5-flash	1	5.000000	
2	react_hybrid	gemini-2.5-pro	20	1.800000	
3	react_research	gemini-2.5-flash	20	1.000000	
4	react_research	gemini-2.5-pro	20	1.000000	
5	self_correcting	gemini-2.5-flash	20	1.150000	
6	self_correcting	gemini-2.5-pro	20	1.100000	
7	self_correcting_hybrid	gemini-2.5-flash	20	1.150000	
8	self_correcting_hybrid	gemini-2.5-pro	20	1.150000	

	min_iterations	max_iterations	avg_cost	cost_per_iteration	\
0	1	5	0.010043	0.005364	
1	5	5	0.036300	0.007260	
2	1	4	0.010286	0.005593	
3	1	1	0.000467	0.000467	
4	1	1	0.000491	0.000491	
5	1	3	0.001791	0.001505	
6	1	2	0.001602	0.001438	
7	1	2	0.006068	0.005034	
8	1	2	0.004661	0.003994	

	termination_reason	termination_pct
0	confidence_reached	95.0
1	max_iterations	5.0
2	confidence_reached	100.0
3	confidence_reached	100.0
4	confidence_reached	100.0
5	validation_passed	100.0
6	validation_passed	100.0
7	validation_passed	100.0
8	validation_passed	100.0

```
[10]: # Multi-turn context growth
context_growth = get_context_growth_analysis()
if not context_growth.empty:
    print("Multi-Turn Context Growth:")
    display(context_growth)

fig = px.line(
    context_growth,
    x='turn',
    y='avg_input_tokens',
    color='model',
    markers=True,
    title='Context Token Growth Across Turns',
```



```

        labels={'turn': 'Turn Number', 'avg_input_tokens': 'Average Input_
↪Tokens'}
    )
    fig.show()

    # Cost growth
    fig2 = px.line(
        context_growth,
        x='turn',
        y='avg_cost',
        color='model',
        markers=True,
        title='Cost Growth Across Turns',
        labels={'turn': 'Turn Number', 'avg_cost': 'Average Cost ($)'}
    )
    fig2.show()

```

Multi-Turn Context Growth:

	pipeline	model	turn	avg_input_tokens	avg_cost
0	multiturn_3	gemini-2.5-flash	1	6.40	0.000761
1	multiturn_3	gemini-2.5-flash	2	2478.20	0.001799
2	multiturn_3	gemini-2.5-flash	3	7172.75	0.001866
3	multiturn_3	gemini-2.5-pro	1	6.40	0.007273
4	multiturn_3	gemini-2.5-pro	2	2850.30	0.012737
5	multiturn_3	gemini-2.5-pro	3	6458.95	0.014743
6	multiturn_5	gemini-2.5-flash	1	6.40	0.000763
7	multiturn_5	gemini-2.5-flash	2	2488.80	0.001042
8	multiturn_5	gemini-2.5-flash	3	4664.60	0.001561
9	multiturn_5	gemini-2.5-flash	4	7477.30	0.001861
10	multiturn_5	gemini-2.5-flash	5	9883.30	0.001801
11	multiturn_5	gemini-2.5-pro	1	6.40	0.007373
12	multiturn_5	gemini-2.5-pro	2	2893.80	0.009993
13	multiturn_5	gemini-2.5-pro	3	5385.95	0.014279
14	multiturn_5	gemini-2.5-pro	4	8345.45	0.017471
15	multiturn_5	gemini-2.5-pro	5	11101.90	0.016804

1.7 6. Streaming Metrics (TTFT)

```

[11]: streaming_data = get_streaming_analysis()
if not streaming_data.empty:
    print("Streaming Metrics:")
    display(streaming_data)

    fig = px.bar(
        streaming_data,
        x='pipeline',
        y='avg_ttft_ms',

```

```

        color='model',
        barmode='group',
        title='Time to First Token (TTFT) by Pipeline',
        labels={'avg_ttft_ms': 'Avg TTFT (ms)', 'pipeline': 'Pipeline'}
    )
    fig.update_layout(xaxis_tickangle=-45)
    fig.show()
else:
    print("No streaming data available. Run experiments with --streaming flag.")

```

Streaming Metrics:

	pipeline	model	stage_type	avg_ttft_ms \
0	context_long	gemini-2.5-flash	evaluation	13181.75
1	context_long	gemini-2.5-flash	extraction	8576.65
2	context_long	gemini-2.5-flash	summarization	9480.00
3	context_long	gemini-2.5-pro	evaluation	20554.00
4	context_long	gemini-2.5-pro	extraction	14013.00
..	...	...	...	...
65	verbosity_cot	gemini-2.5-flash	generation	10368.15
66	verbosity_cot	gemini-2.5-flash	refinement	7688.85
67	verbosity_cot	gemini-2.5-pro	critique	22261.75
68	verbosity_cot	gemini-2.5-pro	generation	20563.90
69	verbosity_cot	gemini-2.5-pro	refinement	19791.35

	avg_total_latency_ms	avg_throughput	samples
0	17621.05	62.4090	20
1	12233.65	80.0225	20
2	11973.55	52.5685	20
3	27674.35	31.1395	20
4	20714.00	42.8815	20
..	...	...	...
65	22337.05	95.2530	20
66	20416.65	106.2050	20
67	33734.55	34.7965	20
68	39282.75	46.3520	20
69	38271.05	49.9625	20

[70 rows x 7 columns]

1.8 7. A/B Testing Analysis

Compare different prompt variants to find optimal prompting strategies.

```

[12]: # Check available A/B tests
ab_tests = get_ab_tests()
if not ab_tests.empty:
    print(f" A/B Tests Conducted: {len(ab_tests)}")

```

```

        display(ab_tests[['test_name', 'prompt_name', 'variants',
↪ 'iterations_per_variant', 'status']])
    else:
        print("No A/B tests found. Run: python -m src.experiment --full-experiment")

```

A/B Tests Conducted: 8

	test_name	prompt_name	\
0	critique_depth	critique	
1	critique_depth	critique	
2	extraction_format	extraction	
3	extraction_format	extraction	
4	validation_strictness	validation	
5	validation_strictness	validation	
6	generation_style	generation	
7	generation_style	generation	

	variants	iterations_per_variant	\
0	["control", "concise", "detailed"]	20	
1	["control", "concise", "detailed"]	20	
2	["control", "structured", "technical"]	20	
3	["control", "structured", "technical"]	20	
4	["control", "strict", "lenient"]	20	
5	["control", "strict", "lenient"]	20	
6	["control", "concise", "detailed", "cot"]	20	
7	["control", "concise", "detailed", "cot"]	20	

	status
0	completed
1	completed
2	completed
3	completed
4	completed
5	completed
6	completed
7	completed

```

[13]: # A/B test cost summary
ab_summary = get_ab_test_summary()
if not ab_summary.empty:
    print("\n A/B Test Cost Summary:")
    display(ab_summary)

    fig = px.bar(
        ab_summary,
        x='prompt_variant',
        y='avg_cost',
        color='model',

```

```

facet_col='ab_test_name',
title='Average Cost by Prompt Variant',
labels={'avg_cost': 'Average Cost ($) ', 'prompt_variant': 'Variant'}
)
fig.show()

```

A/B Test Cost Summary:

	ab_test_name	prompt_variant	model	runs	avg_cost \
0	generation_style	concise	gemini-2.5-flash	20	0.000061
1	generation_style	concise	gemini-2.5-pro	20	0.000675
2	generation_style	control	gemini-2.5-flash	20	0.000887
3	generation_style	cot	gemini-2.5-flash	20	0.001147
4	generation_style	detailed	gemini-2.5-flash	20	0.001539
5	generation_style	control	gemini-2.5-pro	9	0.008772
6	generation_style	cot	gemini-2.5-pro	13	0.009544
7	generation_style	detailed	gemini-2.5-pro	14	0.011832

	avg_latency_ms	avg_input_tokens	avg_output_tokens
0	5093.800000	13.800000	97.900000
1	14728.350000	13.800000	131.500000
2	19135.100000	8.800000	1475.850000
3	20829.600000	30.800000	1904.700000
4	25535.300000	39.800000	2555.600000
5	35862.333333	9.666667	1752.000000
6	38217.538462	31.461538	1900.923077
7	44251.214286	40.285714	2356.428571

```

[14]: # A/B test quality comparison
ab_quality = get_ab_test_quality()
if not ab_quality.empty:
    print("\n A/B Test Quality Summary:")
    display(ab_quality)

    fig = px.bar(
        ab_quality,
        x='prompt_variant',
        y='avg_combined_score',
        color='model',
        facet_col='ab_test_name',
        title='Quality Score by Prompt Variant',
        labels={'avg_combined_score': 'Average Quality Score', 'prompt_variant':
↵ 'Variant'}
    )
    fig.show()

```

A/B Test Quality Summary:

	ab_test_name	prompt_variant	model	runs	\
0	generation_style	concise	gemini-2.5-pro	20	
1	generation_style	concise	gemini-2.5-flash	20	
2	generation_style	control	gemini-2.5-flash	20	
3	generation_style	cot	gemini-2.5-flash	20	
4	generation_style	detailed	gemini-2.5-pro	14	
5	generation_style	cot	gemini-2.5-pro	13	
6	generation_style	control	gemini-2.5-pro	9	
7	generation_style	detailed	gemini-2.5-flash	20	

	avg_automated_score	avg_llm_score	avg_combined_score	\
0	96.000000	9.266500	93.999000	
1	87.500000	9.250500	90.503000	
2	71.322500	9.850500	87.631500	
3	68.728000	9.934000	87.094000	
4	68.045000	9.976429	87.077143	
5	68.390769	9.923846	86.898462	
6	68.320000	9.926667	86.887778	
7	67.530500	9.967000	86.814500	

	avg_response_length	avg_vocabulary_richness
0	584.000000	0.724220
1	461.050000	0.810197
2	6716.900000	0.427474
3	8542.750000	0.384551
4	10295.714286	0.360868
5	8093.615385	0.367818
6	7227.777778	0.388628
7	11734.600000	0.360604

```
[15]: # Cost-quality ratio (lower is better = more efficient)
ab_ratio = get_ab_test_cost_quality_ratio()
if not ab_ratio.empty:
    print("\n Cost-Quality Ratio (lower = better value):")
    display(ab_ratio.sort_values('cost_per_quality_point'))

    fig = px.scatter(
        ab_ratio,
        x='avg_cost',
        y='avg_quality',
        color='prompt_variant',
        symbol='model',
        size='runs',
        title='Cost vs Quality by Prompt Variant',
        labels={'avg_cost': 'Average Cost ($)'}, 'avg_quality': 'Quality Score'},
        hover_data=['ab_test_name', 'runs']
    )
```

```
fig.show()
```

Cost-Quality Ratio (lower = better value):

	ab_test_name	prompt_variant	model	runs	avg_cost	\
0	generation_style	concise	gemini-2.5-flash	20	0.000061	
1	generation_style	concise	gemini-2.5-pro	20	0.000675	
2	generation_style	control	gemini-2.5-flash	20	0.000887	
3	generation_style	cot	gemini-2.5-flash	20	0.001147	
4	generation_style	detailed	gemini-2.5-flash	20	0.001539	
5	generation_style	control	gemini-2.5-pro	9	0.008772	
6	generation_style	cot	gemini-2.5-pro	13	0.009544	
7	generation_style	detailed	gemini-2.5-pro	14	0.011832	

	avg_quality	cost_per_quality_point
0	90.503000	0.066610
1	93.999000	0.717276
2	87.631500	1.012207
3	87.094000	1.319363
4	86.814500	1.775368
5	86.887778	10.100845
6	86.898462	10.983885
7	87.077143	13.591049

```
[16]: # Statistical significance test between variants
if not runs_df.empty and 'prompt_variant' in runs_df.columns:
    ab_runs = runs_df[runs_df['ab_test_name'].notna()]
    if len(ab_runs) > 0:
        variants = ab_runs['prompt_variant'].unique()
        if len(variants) >= 2:
            print("\n Statistical Significance Tests:")
            for i, v1 in enumerate(variants):
                for v2 in variants[i+1:]:
                    costs_v1 = ab_runs[ab_runs['prompt_variant'] == v1]
                    costs_v2 = ab_runs[ab_runs['prompt_variant'] == v2]
                    if len(costs_v1) > 5 and len(costs_v2) > 5:
                        t_stat, p_value = stats.ttest_ind(costs_v1, costs_v2)
                        sig = '***' if p_value < 0.001 else '**' if p_value < 0.01 else '*' if p_value < 0.05 else ''
                        print(f" {v1} vs {v2}: p={p_value:.4f} {sig}")
```

Statistical Significance Tests:
control vs concise: p=0.0000 ***
control vs detailed: p=0.0392 *

```
control vs cot: p=0.2850
concise vs detailed: p=0.0000 ***
concise vs cot: p=0.0000 ***
detailed vs cot: p=0.2638
```

1.9 8. Document Analysis & Vulnerability Detection

Analyze how well different pipelines detect security vulnerabilities.

```
[17]: # Ground truth vulnerability counts
print(" Vulnerability Ground Truth Summary:")
print("="*50)

doc_ids = get_all_document_ids()
gt_summary = []

for doc_id in doc_ids:
    vulns = get_vulnerabilities(doc_id)
    severity_counts = count_by_severity(doc_id)
    gt_summary.append({
        'document': vulns.document_name[:40],
        'total_vulns': vulns.total_count,
        'critical': severity_counts.get('critical', 0),
        'high': severity_counts.get('high', 0),
        'medium': severity_counts.get('medium', 0),
        'low': severity_counts.get('low', 0),
    })

gt_df = pd.DataFrame(gt_summary)
display(gt_df)

print(f"\nTotal vulnerabilities across all documents: {gt_df['total_vulns'].
    ↪sum()}")
```

Vulnerability Ground Truth Summary:

```
=====
```

	document	total_vulns	critical	high	\
0	User Authentication Module (user_auth.py	14	4	6	
1	Flask REST API (api_server.py)	18	6	6	
2	Kubernetes Deployment (kubernetes-deploy	18	10	3	
3	AWS Terraform (aws-infrastructure.tf)	20	7	6	
4	Bank Login Page (login-page.html)	15	6	5	
5	Application Config (app-config.json)	20	8	8	
6	Application Dockerfile	15	5	3	
7	Docker Compose (docker-compose.yaml)	17	7	9	
8	Architecture Spec (architecture-spec.md)	25	14	8	

```

medium low

```

0	4	0
1	3	3
2	5	0
3	6	1
4	3	1
5	4	0
6	3	4
7	1	0
8	3	0

Total vulnerabilities across all documents: 162

```
[18]: # Severity distribution visualization
severity_totals = gt_df[['critical', 'high', 'medium', 'low']].sum()

fig = px.pie(
    values=severity_totals.values,
    names=severity_totals.index,
    title='Vulnerability Distribution by Severity',
    color=severity_totals.index,
    color_discrete_map={
        'critical': '#dc3545',
        'high': '#fd7e14',
        'medium': '#ffc107',
        'low': '#28a745'
    }
)
fig.show()
```

```
[19]: # Document workflow analysis
doc_runs = runs_df[runs_df['workflow'] == 'document']
if not doc_runs.empty:
    print(" Document Analysis Pipeline Comparison:")

    doc_summary = doc_runs.groupby(['pipeline', 'model']).agg({
        'total_cost': ['mean', 'std'],
        'total_latency_ms': 'mean',
        'num_stages': 'first',
        'id': 'count'
    }).round(4)
    doc_summary.columns = ['avg_cost', 'std_cost', 'avg_latency', 'stages', 'runs']
    doc_summary = doc_summary.reset_index()
    display(doc_summary)

    fig = px.bar(
        doc_summary,
```



```

        x='pipeline',
        y='avg_cost',
        color='model',
        barmode='group',
        error_y='std_cost',
        title='Document Analysis Pipeline Costs',
        labels={'avg_cost': 'Average Cost ($)', 'pipeline': 'Pipeline'}
    )
    fig.update_layout(xaxis_tickangle=-45)
    fig.show()
else:
    print("No document workflow data. Run: python -m src.experiment --workflow_
    ↪document --model flash")

```

Document Analysis Pipeline Comparison:

	pipeline	model	avg_cost	std_cost	avg_latency \
0	doc_analysis_hybrid	gemini-2.5-flash	0.0177	0.0020	95427.50
1	doc_analysis_hybrid	gemini-2.5-pro	0.0176	0.0018	95151.25
2	doc_analysis_iterative	gemini-2.5-flash	0.0143	0.0018	97717.70
3	doc_analysis_iterative	gemini-2.5-pro	0.0300	0.0029	123374.75
4	doc_analysis_simple	gemini-2.5-flash	0.0022	0.0004	37953.00
5	doc_analysis_simple	gemini-2.5-pro	0.0148	0.0023	63401.70
6	doc_analysis_thorough	gemini-2.5-flash	0.0167	0.0035	145486.80
7	doc_analysis_thorough	gemini-2.5-pro	0.0467	0.0050	176434.80

	stages	runs
0	3	20
1	3	20
2	3	20
3	3	20
4	2	20
5	2	20
6	4	20
7	4	20

1.10 9. Cost-Quality Tradeoffs

```

[20]: # Merge runs with quality scores
if not quality_df.empty and not runs_df.empty:
    runs_quality = runs_df.merge(quality_df, left_on='id', right_on='run_id',
    ↪how='inner')

    if not runs_quality.empty:
        print(f"Runs with quality scores: {len(runs_quality)}")

        # Cost vs Quality scatter
        fig = px.scatter(

```

```

        runs_quality,
        x='total_cost',
        y='combined_score',
        color='pipeline',
        symbol='model',
        title='Cost vs Quality Score',
        labels={'total_cost': 'Cost ($)', 'combined_score': 'Quality Score'}
        # trendline='ols'
    )
    fig.show()

    # Calculate cost efficiency (quality per dollar)
    runs_quality['quality_per_dollar'] = runs_quality['combined_score'] /
    ↪(runs_quality['total_cost'] * 1000 + 0.001)

    efficiency = runs_quality.groupby(['pipeline', 'model']).agg({
        'quality_per_dollar': 'mean',
        'combined_score': 'mean',
        'total_cost': 'mean'
    }).round(2)
    efficiency = efficiency.sort_values('quality_per_dollar',
    ↪ascending=False)
    print("\n Cost Efficiency Ranking (quality per dollar, higher is
    ↪better):")
    display(efficiency)
else:
    print("Quality data not available. Run experiments with --llm-eval flag.")

```

Runs with quality scores: 735

Cost Efficiency Ranking (quality per dollar, higher is better):

pipeline	model	quality_per_dollar	combined_score \
verbosity_concise	gemini-2.5-flash	2420.67	86.90
context_short	gemini-2.5-flash	606.62	90.71
ab_generation	gemini-2.5-flash	563.41	88.01
verbosity_concise	gemini-2.5-pro	295.98	84.80
react_research	gemini-2.5-flash	220.23	92.86
	gemini-2.5-pro	214.80	91.13
context_short	gemini-2.5-pro	81.66	90.22
self_correcting	gemini-2.5-flash	66.21	85.12
ab_generation	gemini-2.5-pro	65.52	89.48
self_correcting	gemini-2.5-pro	65.50	84.85
doc_analysis_simple	gemini-2.5-flash	39.81	85.82
context_long	gemini-2.5-flash	39.31	78.82
verbosity_cot	gemini-2.5-flash	24.86	87.47

self_correcting_hybrid	gemini-2.5-pro	21.18	85.69
	gemini-2.5-flash	18.81	86.01
react_hybrid	gemini-2.5-flash	13.12	87.45
	gemini-2.5-pro	11.75	89.58
multiturn_3	gemini-2.5-flash	10.86	46.38
multiturn_5	gemini-2.5-flash	8.64	59.99
hybrid_cot	gemini-2.5-flash	7.89	86.84
	gemini-2.5-pro	7.78	87.32
context_long	gemini-2.5-pro	6.26	87.98
doc_analysis_simple	gemini-2.5-pro	6.11	88.13
doc_analysis_iterative	gemini-2.5-flash	6.06	85.45
doc_analysis_thorough	gemini-2.5-flash	5.19	83.57
doc_analysis_hybrid	gemini-2.5-pro	4.74	82.64
	gemini-2.5-flash	4.65	82.75
verbosity_cot	gemini-2.5-pro	3.10	86.92
doc_analysis_iterative	gemini-2.5-pro	2.94	87.44
doc_analysis_thorough	gemini-2.5-pro	1.81	83.58
multiturn_3	gemini-2.5-pro	1.44	49.50
multiturn_5	gemini-2.5-pro	1.15	75.71

		total_cost
pipeline	model	
verbosity_concise	gemini-2.5-flash	0.00
context_short	gemini-2.5-flash	0.00
ab_generation	gemini-2.5-flash	0.00
verbosity_concise	gemini-2.5-pro	0.00
react_research	gemini-2.5-flash	0.00
	gemini-2.5-pro	0.00
context_short	gemini-2.5-pro	0.00
self_correcting	gemini-2.5-flash	0.00
ab_generation	gemini-2.5-pro	0.01
self_correcting	gemini-2.5-pro	0.00
doc_analysis_simple	gemini-2.5-flash	0.00
context_long	gemini-2.5-flash	0.00
verbosity_cot	gemini-2.5-flash	0.00
self_correcting_hybrid	gemini-2.5-pro	0.00
	gemini-2.5-flash	0.01
react_hybrid	gemini-2.5-flash	0.01
	gemini-2.5-pro	0.01
multiturn_3	gemini-2.5-flash	0.00
multiturn_5	gemini-2.5-flash	0.01
hybrid_cot	gemini-2.5-flash	0.01
	gemini-2.5-pro	0.01
context_long	gemini-2.5-pro	0.01
doc_analysis_simple	gemini-2.5-pro	0.01
doc_analysis_iterative	gemini-2.5-flash	0.01
doc_analysis_thorough	gemini-2.5-flash	0.02
doc_analysis_hybrid	gemini-2.5-pro	0.02

	gemini-2.5-flash	0.02
verbosity_cot	gemini-2.5-pro	0.03
doc_analysis_iterative	gemini-2.5-pro	0.03
doc_analysis_thorough	gemini-2.5-pro	0.05
multiturn_3	gemini-2.5-pro	0.03
multiturn_5	gemini-2.5-pro	0.07

1.11 10. Key Findings & Recommendations

```
[21]: print("\n" + "="*60)
print(" KEY FINDINGS SUMMARY")
print("="*60)

if not runs_df.empty:
    # Most expensive pipeline
    pipeline_costs = runs_df.groupby('pipeline')['total_cost'].mean().
    ↪sort_values(ascending=False)
    print(f"\n Most expensive pipeline: {pipeline_costs.index[0]}_
    ↪({format_cost(pipeline_costs.iloc[0])}/run)")
    print(f" Cheapest pipeline: {pipeline_costs.index[-1]}_
    ↪({format_cost(pipeline_costs.iloc[-1])}/run)")

    # Model comparison
    model_avg = runs_df.groupby('model')['total_cost'].mean()
    if len(model_avg) >= 2:
        ratio = model_avg.max() / model_avg.min()
        print(f"\n Model cost ratio: {ratio:.1f}x ({model_avg.idxmax()} vs_
        ↪{model_avg.idxmin()}")

    # Total spent
    total = runs_df['total_cost'].sum()
    print(f"\n Total experiment cost: {format_cost(total)}")
    print(f" Total runs: {len(runs_df):,}")
    print(f" Average cost per run: {format_cost(total/len(runs_df))}")

if not ab_summary.empty:
    print("\n A/B Test Insights:")
    best_variant = ab_summary.loc[ab_summary['avg_cost'].idxmin()]
    print(f" Most cost-efficient variant: {best_variant['prompt_variant']}")

print("\n" + "="*60)
```

```
=====
KEY FINDINGS SUMMARY
=====
```

Most expensive pipeline: multiturn_5 (\$0.0365/run)

Cheapest pipeline: verbosity_concise (\$0.000162/run)

Model cost ratio: 3.1x (gemini-2.5-pro vs gemini-2.5-flash)

Total experiment cost: \$8.07

Total runs: 736

Average cost per run: \$0.0110

A/B Test Insights:

Most cost-efficient variant: concise

=====

```
[22]: # Export summary data
print("\n Exporting summary data...")

# Save to CSV for external analysis
if not runs_df.empty:
    runs_df.to_csv('../data/runs_export.csv', index=False)
    print("    Saved: data/runs_export.csv")

if not stages_df.empty:
    stages_df.to_csv('../data/stages_export.csv', index=False)
    print("    Saved: data/stages_export.csv")

if not ab_summary.empty:
    ab_summary.to_csv('../data/ab_tests_export.csv', index=False)
    print("    Saved: data/ab_tests_export.csv")

print("\n Analysis complete!")
```

Exporting summary data...

Saved: data/runs_export.csv

Saved: data/stages_export.csv

Saved: data/ab_tests_export.csv

Analysis complete!